# SPRING BOOT

Q.1] What is Spring Framework?
Q.2] Why do we use Spring?
Q.3] What is SpringBoot?
Q.4] Why do we use SpringBoot?
Q.5] Diff. between Spring & SpringBoot.
Q.6] What is Spring Annotation?
Q.7] @SpringBootApplication ?
Q.8] IoC vs Dependency Injection
Q.9] Types of Dependency Injection.
Q.10] What is Spring Bean?
Q.11] SpringBean Lifecycle
Q.12] What is SpringBoot Actuator?
Q.13] @Controller vs @RestController
Q.14] @RequestBody vs @ResponseBody
Q.15] @Component vs @service vs @Repository
Q.16] @Autowired vs @Constructor Injection
Q.17] @Primary vs @Qualifier
Q.18] What is Spring Security?
Q.19] What is OAuth 2.0 in Spring Security?
Q.20] @Transactional ? How does it work?
When should we use it?
Q.21] @Values vs @ConfigurationProperties

Q. 1] What is Spring Framework ?

⇒ ① Spring is a Java Framework used to develop enterprise applications such as web applications, REST APIs and backend systems.

② Spring framework provides features like Dependency injection (DI), Security, database integration.

Q. 2] Why do we use Spring?

⇒ ① Without framework, we need to manage object creation, dependencies and configurations manually. Spring helps to manage these tasks and make applications easier to maintain & test.

② eg:- Suppose, we are developing an employee management system.

1) Controller handles incoming requests.
2) Service contains buisiness logic.
3) Repository communicates with database.
4) Spring manages the objects and their dependancies

Q. 3] What is Spring Boot ?

⟶ ① Spring Boot is a built on top of the spring framework.

② It simplifies Spring application development by reducing manual configuration.

Q.4] Why do we use Spring Boot?

⇒ ① Spring applications can required considerable setup. but spring boot provides →

1) Auto Configuration : Configures components based on dependencies and settings.

2) Starter dependencies : Make it easier to add related libraries.

3) Embedded server : Run web applications using servers such as Tomcat without requiring a seperately installed server.

4) Production support : Provide tools such as SpringBoot Actuator for health checks and monitoring.

Q.5]

| Spring | SpringBoot |
|---|---|
| ① Requires more configuration. | ① Reduces configurations through auto-configuration. |
| ② Dependencies are managed individually. | ② Starter dependencies simplify dependency management. |
| ③ Server setup may be manual. | ③ Includes embedded server such as tomcat. |
| ④ Take more setup to create an applications. | ④ Helps build applications quickly. |

Q.6] What is a Spring Annotation ?

⇒ ① Spring annotation are special markers in JAVA that provides instructions to the spring framework & reduce boilerplate configuration.

② They help to Spring to manage applications components, identify classes, methods without requiring extensive XML configuration.

③ Annotations start with @ symbol.

④ eg - 1) @ Component 2) @ Service
3) @ Repository 4) @ Controller
5) @ RestController 6) @ Autowired
7) @ Configuration 8) @ Bean
9) @ SpringBootApplication
10) @ GetMapping 11) @ PostMapping

Q.7] What is @SpringBootApplication ?

⇒ ① It is main annotation used to start a SpringBoot application. It combines three annotations.

a) @ SpringBootConfiguration
b) @ EnableAutoConfiguration
c) @ ComponentScan.

Q. 8] IoC vs Dependency Injection (DI)

⇒ ① IoC (Inversion of Control) : The Spring container manages object creation and their lifecycle instead of developer managing everything manually.

② Dependency Injection : A technique through which Spring ~~boot~~ provides the dependencies an object needs.

Q. 9] What are the type of Dependency injection?

⇒ ① Constructor Injection → Dependencies are passed through a constructor.

② Setter Injection → Dependencies are provided through setter methods.

③ Field Injections → Dependencies are injected directly into fields using @Autowired.

④ For best practice prefer constructor injections beacause it provides immutability & make unit testing easier.

Q. 10] What is Spring Bean?

⇒ ① Spring Bean is an object that is created, configured and managed by Spring IOC container instead of manually.

Q.11] Explain Spring Bean Lifecycle.

⟹ ① Spring creates the bean.
② Spring injects the dependencies.
③ Initialization callback's run, such as @PostConstruct.
④ The bean is available for use.
⑤ When container shuts down, destructions callbacks run, such as @PreDestroy.

---

Q.12] What is Spring Boot Actuator?

⟹ ① SpringBoot Actuator provides endpoints for monitoring and managing an applications.

② Common endpoints :
a) /actuator/health ⇒ Check application health.
b) /actuator/info ⇒ display applicat$^{n}$ information.
c) /actuator/metrics ⇒ expose applicat$^{n}$ metrics.

---

Q.13] @Controller vs @RestController

⟹ ① @Controller is primarily used for MVC applications that returns views such as HTML pages.

② @RestController is used for REST APIs and returns response data, usually JSON.

---

Q.14] @RequestBody vs @ResponseBody

⟹ ① @RequestBody convers incoming request data such as JSON into Java object.

② @ResponseBody converts a method's return value into HTTP response Body, commonly JSON.

Q.15] @Component vs @Service vs @Repository

⇒ All three anotations allow to Spring to detect and register classes as beans.

1) @Component → general-purpose Spring-managed component.

2) @Service → represent business logic.

3) @Repository → represent the data access layer and supports persistence exception translation.

---

Q.16] @Autowired vs Constructor Injection

⇒ ① @Autowired tells Spring to inject a dependency into a supported injection point.

② Constructor injection provides dependencies through the class constructor.

③ If a class has only one constructor, Spring can inject its dependencies without @Autowired.

④ Best practice to use constructor injection rather than field injection because dependencies are explicit, fields can be final and testing is easier.

*enlivo

⑤ @Service
class UserService {
private final UserRepository repository;

@Autowired
public UserService(UserRepository repository){
this.repository = repository;
}
}

---

Q.17] @Primary Vs @Qualifier

⇒ ① Those annotations resolve ambiguity when multiple beans of the same type exist.

a) @Primary specifies the default bean when multiple candidates are available.

b) @Qualifier selects a particular been by its qualifier.

---

Q.18] What is Spring Security?

→ ① Spring security is a framework that protects Spring applications by providing authentication authorization and other security features.

② It supports :
a) Form-based login
b) Role-based access control
c) JWT-based authentication
d) OAuth 2.0 and OpenID connect.
e) Password encoding & CSRF protection.

③ eg - A public API may be accessible without login, while an admin API requires authentication and an appropriate role.

Q.19] What is OAuth 2.0 in Spring Security?

⇒ ① OAuth 2.0 is an authorization framework that allows an application to obtain limited access to resources without directly handling the user's password for the external service.

Q.20] What is @Transactional? How does it work internally, and when should we use it?

⇒ ① @Transactional defines a transaction boundary for a method or class, allowing related database operations to commit together or roll back on qualifying failures.

② How it works internally :-

a) Spring detects the @Transactional annotation.
b) Spring's transaction infrastructure typically uses a proxy around the bean.
c) The proxy starts or joins a transaction before invoking the method.
d) On successful completion, the transaction commits.
e) On a qualifying exception, it rolls back according to configured rollback rules.

③ We use it —
a) Transfering money between accounts.
b) Saving an order and its order items.
c) Updating multiple related database records together.

---

Q. 21] @Value Vs @ConfigurationProperties.

⇒ ① @Value injects an individual property
② @ConfigurationProperties binds a group of related properties to a Java class.

③ @Value convenient for a few value.
④ @ConfigurationProperties is better for multiple related configuration values.

⑤ No dedicated type-safe configuration class is required in @Value
⑥ @ConfigurationProperties provide structured, type-safe configuration ~~but~~ binding.

⑦ @Value: Reading a single application name or API URL.
⑧ @ConfigurationProperties: Reading database setting, email setting or multiple payment-service configurations.